

СОДЕРЖАНИЕ

Введение	2
1. Постановка задачи.....	3
2. Порождающая грамматика	4
2.1 Классификация грамматики	5
2.2 Анализ выбранной грамматики	7
2.2.1 Проверка достижимости терминалов	7
2.2.2 Проверка продуктивности нетерминалов	8
2.2.3 Проверка ε -продукций	9
2.2.4 Проверка левой рекурсии	10
3. Детекция синтаксических ошибок	12
3.1 Метод Айронса	14
4. Программная реализация	16
5. Практические примеры.....	19
Заключение.....	20
Список использованных источников.....	21
Приложение А.....	22

Введение

Разработка языковых процессоров связана с формальным описанием структуры исходного текста и последующей проверкой его соответствия заданным правилам языка. Одним из ключевых этапов такой обработки является синтаксический анализ, позволяющий определить, образует ли последовательность лексем корректную конструкцию и в какой точке возникает нарушение синтаксиса.

Для построения синтаксического анализатора используется формальная грамматика, задающая допустимую структуру исследуемого фрагмента языка. От выбора грамматики зависят возможности распознавания, сложность реализации анализатора, способ обработки вложенных конструкций и механизм восстановления после ошибок. Поэтому важным этапом является не только разработка грамматики, но и анализ её свойств: класса по Хомскому, рекурсивности, продуктивности правил и однозначности разбора.

Практическая ценность синтаксического анализатора определяется также качеством диагностики ошибок. Простого обнаружения некорректной конструкции недостаточно: желательно определить место нарушения, сформировать понятное диагностическое сообщение и, по возможности, продолжить разбор оставшейся части текста. Для этого применяются методы нейтрализации синтаксических ошибок.

В данной работе рассматривается построение и программная реализация анализатора для оператора цикла `while` языка C++. Основное внимание уделяется формальному описанию синтаксиса, исследованию свойств выбранной грамматики, организации лексического и синтаксического анализа, а также обработке и нейтрализации обнаруженных ошибок.

1. Постановка задачи

Требуется разработать язык и порождающую грамматику оператора цикла `while` языка `C++`. Выполнить программную реализацию алгоритма синтаксического анализа (парсер).

Необходимо разработать кроссплатформенное приложение с графическим интерфейсом.

2. Порождающая грамматика

Определим грамматику оператора цикла `while` языка C++ $G[<While>]$ в расширенной форме Бэкуса–Наура (EBNF) с продукциями P:

1. $<While> \rightarrow \text{while } (<Condition>) <Body>$
2. $<Body> \rightarrow <Block> \mid <Expression> ";" \mid <While>$
3. $<Block> \rightarrow "{" \{ <Statement> \} "}"$
4. $<Statement> \rightarrow <Expression> ";" \mid <While> \mid \text{break ";" } \mid \text{continue ";"}$
5. $<Condition> \rightarrow <LogicTerm> \{ \mid <LogicTerm> \}$
6. $<LogicTerm> \rightarrow <Relation> \{ \&\& <Relation> \}$
7. $<Relation> \rightarrow [!] <Arithmetic> [(== \mid != \mid < \mid > \mid <= \mid >=) <Arithmetic>]$
8. $<Expression> \rightarrow <Assignment> \mid <Arithmetic> \mid ++ <Identifier>$
 $\mid -- <Identifier> \mid <Identifier> ++$
 $\mid <Identifier> --$
9. $<Assignment> \rightarrow <Identifier> (= \mid += \mid -= \mid *= \mid /= \mid \% =) <Arithmetic>$
10. $<Arithmetic> \rightarrow <Term> \{ (+ \mid -) <Term> \}$
11. $<Term> \rightarrow <Factor> \{ (* \mid / \mid \%) <Factor> \}$
12. $<Factor> \rightarrow <Identifier> \mid <Number> \mid \text{true} \mid \text{false} \mid (<Condition>)$
13. $<Identifier> \rightarrow <Letter> \{ <Letter> \mid <Digit> \mid _ \}$
14. $<Number> \rightarrow <Digit> \{ <Digit> \}$
15. $<Letter> \rightarrow "a" \mid "b" \mid \dots \mid "z" \mid "A" \mid "B" \mid \dots \mid "Z"$
16. $<Digit> \rightarrow "0" \mid "1" \mid "2" \mid "3" \mid "4" \mid "5" \mid "6" \mid "7" \mid "8" \mid "9"$

В используемой записи символ `"|"` обозначает выбор одной из альтернатив, квадратные скобки `"[]"` – необязательную часть конструкции, а фигурные скобки `"{}"` – повторение заключённой в них конструкции ноль или более раз. Кавычки в правиле `<Block>` используются для обозначения непосредственно терминальных символов `"{"` и `"}"` языка C++.

Начальным символом грамматики является нетерминал `<While>`.

Следуя формальному определению грамматики, представим $G[\langle \text{While} \rangle]$ её составляющими:

- $Z = \langle \text{While} \rangle$;
- $V_N = \{ \langle \text{While} \rangle, \langle \text{Body} \rangle, \langle \text{Block} \rangle, \langle \text{Statement} \rangle, \langle \text{Condition} \rangle, \langle \text{LogicTerm} \rangle, \langle \text{Relation} \rangle, \langle \text{Expression} \rangle, \langle \text{Assignment} \rangle, \langle \text{Arithmetic} \rangle, \langle \text{Term} \rangle, \langle \text{Factor} \rangle, \langle \text{Identifier} \rangle, \langle \text{Number} \rangle, \langle \text{Letter} \rangle, \langle \text{Digit} \rangle \}$;
- $V_T = \{ \text{while, break, continue, true, false, (,), \{, \}, :, ||, \&\&, !, ==, !=, <, >, <=, >=, =, +=, -=, *=, /=, \% =, +, -, *, /, \%, ++, --, _, a, ..., z, A, ..., Z, 0, ..., 9} \}$.

Таким образом, формально грамматика задаётся четвёркой:

$$G = (V_T, V_N, P, Z),$$

где V_T – множество терминальных символов, V_N – множество нетерминальных символов, P – множество продукций грамматики, Z – начальный символ.

Основная конструкция цикла определяется правилом:

$$\langle \text{While} \rangle \rightarrow \text{while} (\langle \text{Condition} \rangle) \langle \text{Body} \rangle$$

В соответствии с данным правилом оператор состоит из ключевого слова `while`, условия, заключённого в круглые скобки, и тела цикла.

Таким образом, заданная грамматика определяет синтаксические конструкции, необходимые для анализа оператора цикла `while`, его условий, арифметических и логических выражений, операций присваивания, инкремента и декремента, а также операторов `break` и `continue`.

2.1 Классификация грамматики

Согласно классификации Хомского, грамматика $G[\langle \text{While} \rangle]$ относится к классу контекстно-свободных грамматик, то есть является грамматикой типа 2.

Контекстно-свободной называется грамматика, продукции которой в обычной форме Бэкуса–Наура могут быть представлены в виде: $A \rightarrow \alpha$, где A

– один нетерминальный символ, а α – произвольная последовательность терминальных и нетерминальных символов.

Разработанная грамматика записана в расширенной форме Бэкуса–Наура EBNF. Конструкции [] и { }, используемые в EBNF, являются сокращённой формой записи и при необходимости могут быть преобразованы в обычные продукции BNF путём введения дополнительных нетерминальных символов.

Например, правило:

$$\langle \text{Condition} \rangle \rightarrow \langle \text{LogicTerm} \rangle \{ \parallel \langle \text{LogicTerm} \rangle \}$$

может быть представлено в обычной форме следующим образом:

$$\langle \text{Condition} \rangle \rightarrow \langle \text{LogicTerm} \rangle \langle \text{ConditionTail} \rangle$$
$$\langle \text{ConditionTail} \rangle \rightarrow \parallel \langle \text{LogicTerm} \rangle \langle \text{ConditionTail} \rangle \mid \varepsilon$$

А правило:

$$\langle \text{Arithmetic} \rangle \rightarrow \langle \text{Term} \rangle \{ (+ \mid -) \langle \text{Term} \rangle \}$$

может быть представлено как:

$$\langle \text{Arithmetic} \rangle \rightarrow \langle \text{Term} \rangle \langle \text{ArithmeticTail} \rangle$$
$$\langle \text{ArithmeticTail} \rangle \rightarrow + \langle \text{Term} \rangle \langle \text{ArithmeticTail} \rangle \mid - \langle \text{Term} \rangle \langle \text{ArithmeticTail} \rangle \mid \varepsilon$$

Во всех полученных продукциях слева находится только один нетерминальный символ. Следовательно, грамматика удовлетворяет определению контекстно-свободной грамматики.

При этом грамматика не относится к регулярным, или автоматным, грамматикам типа 3.

Для праволинейной регулярной грамматики продукции должны иметь форму $A \rightarrow Ab$, $A \rightarrow a$ или $A \rightarrow \varepsilon$, где A и B – нетерминальные символы, а a – терминальный символ.

В разработанной грамматике имеются правила, которые не соответствуют данной форме, например:

$$\langle \text{While} \rangle \rightarrow \text{while} (\langle \text{Condition} \rangle) \langle \text{Body} \rangle$$

Правило содержит несколько терминальных символов и несколько нетерминальных составляющих. Аналогично правило

$$\langle \text{Factor} \rangle \rightarrow (\langle \text{Condition} \rangle)$$
 содержит нетерминальный символ между двумя

терминальными символами. Кроме того, грамматика допускает рекурсивное вложение конструкций. В частности, $\langle \text{Body} \rangle \rightarrow \langle \text{While} \rangle$ и $\langle \text{Statement} \rangle \rightarrow \langle \text{While} \rangle$ позволяют помещать оператор `while` внутрь другого оператора `while`.

Пример: `while (x < 10) while (y < 5) y++;`

Наличие подобных рекурсивных и вложенных конструкций характерно для контекстно-свободных языков и не описывается праволинейными или леволинейными продукциями регулярной грамматики.

Таким образом, разработанная грамматика $G[\langle \text{While} \rangle]$ является контекстно-свободной грамматикой типа 2 по классификации Хомского.

2.2 Анализ выбранной грамматики

Для оценки формальных свойств выбранной грамматики $G[\langle \text{While} \rangle]$ проведён её анализ по следующим критериям: достижимость и продуктивность нетерминальных символов, наличие ϵ -продукций и левой рекурсии.

Грамматика записана в расширенной форме Бэкуса–Наура (EBNF). Поэтому конструкции $\{ \dots \}$ и $[\dots]$ при проведении формального анализа рассматриваются как сокращённая запись повторяющихся и необязательных фрагментов соответственно. При необходимости они могут быть преобразованы в обычные продукции BNF посредством введения вспомогательных нетерминалов. Такое преобразование не изменяет язык, задаваемый грамматикой.

2.2.1 Проверка достижимости терминалов

Нетерминальный символ называется достижимым, если он может встретиться хотя бы в одной сентенциальной форме, полученной из начального символа грамматики. Наличие недостижимых нетерминалов

свидетельствует о существовании правил, которые не могут быть использованы при построении ни одной допустимой входной цепочки.

Начальным символом рассматриваемой грамматики является $\langle \text{While} \rangle$. Из правила $\langle \text{While} \rangle \rightarrow \text{while} (\langle \text{Condition} \rangle) \langle \text{Body} \rangle$ непосредственно достигаются нетерминалы $\langle \text{Condition} \rangle$ и $\langle \text{Body} \rangle$. Из $\langle \text{Condition} \rangle$ последовательно достигаются $\langle \text{Condition} \rangle \rightarrow \langle \text{LogicTerm} \rangle \rightarrow \langle \text{Relation} \rangle \rightarrow \langle \text{Arithmetic} \rangle \rightarrow \langle \text{Term} \rangle \rightarrow \langle \text{Factor} \rangle$. Из $\langle \text{Factor} \rangle$ достигаются $\langle \text{Identifier} \rangle$ и $\langle \text{Number} \rangle$, а из них, в свою очередь, – $\langle \text{Letter} \rangle$ и $\langle \text{Digit} \rangle$. Из нетерминала $\langle \text{Body} \rangle$ достигаются $\langle \text{Block} \rangle$, $\langle \text{Expression} \rangle$ и повторно $\langle \text{While} \rangle$. Из $\langle \text{Block} \rangle$ достигается $\langle \text{Statement} \rangle$. Из $\langle \text{Expression} \rangle$ достигается $\langle \text{Assignment} \rangle$, а также уже рассмотренные $\langle \text{Arithmetic} \rangle$ и $\langle \text{Identifier} \rangle$.

Таким образом, из начального символа $\langle \text{While} \rangle$ достижимы все нетерминалы грамматики: $\langle \text{While} \rangle$, $\langle \text{Body} \rangle$, $\langle \text{Block} \rangle$, $\langle \text{Statement} \rangle$, $\langle \text{Condition} \rangle$, $\langle \text{LogicTerm} \rangle$, $\langle \text{Relation} \rangle$, $\langle \text{Expression} \rangle$, $\langle \text{Assignment} \rangle$, $\langle \text{Arithmetic} \rangle$, $\langle \text{Term} \rangle$, $\langle \text{Factor} \rangle$, $\langle \text{Identifier} \rangle$, $\langle \text{Number} \rangle$, $\langle \text{Letter} \rangle$, $\langle \text{Digit} \rangle$.

Недостижимые нетерминальные символы в выбранной грамматике отсутствуют. Все введенные правила могут участвовать в построении допустимых цепочек языка.

2.2.2 Проверка продуктивности нетерминалов

Нетерминальный символ называется продуктивным, если из него за конечное число применений продукций может быть выведена цепочка, состоящая только из терминальных символов. Непродуктивный нетерминал не способен участвовать в построении законченной входной цепочки.

Проверку продуктивности целесообразно выполнять снизу вверх, начиная с нетерминалов, непосредственно порождающих терминальные символы. Для $\langle \text{Letter} \rangle$ и $\langle \text{Digit} \rangle$ продуктивность следует непосредственно из правил:

$$\langle \text{Letter} \rangle \rightarrow "a" \mid "b" \mid \dots \mid "z" \mid "A" \mid \dots \mid "Z"$$
$$\langle \text{Digit} \rangle \rightarrow "0" \mid "1" \mid \dots \mid "9".$$

Следовательно, продуктивны $\langle \text{Identifier} \rangle$ и $\langle \text{Number} \rangle$.

Далее продуктивность последовательно распространяется на $\langle \text{Factor} \rangle$, $\langle \text{Term} \rangle$ и $\langle \text{Arithmetic} \rangle$, исходя из правил вывода. Из этого следует продуктивность $\langle \text{Relation} \rangle$, $\langle \text{LogicTerm} \rangle$ и $\langle \text{Condition} \rangle$. Нетерминал $\langle \text{Assignment} \rangle$ также является продуктивным, так как состоит из $\langle \text{Identifier} \rangle$ и $\langle \text{Arithmetic} \rangle$.

Для $\langle \text{Expression} \rangle$ существует несколько конечных выводов, например $\langle \text{Expression} \rangle \rightarrow \langle \text{Arithmetic} \rangle$. Продуктивность $\langle \text{Statement} \rangle$ следует из продукции $\langle \text{Statement} \rangle \rightarrow \langle \text{Expression} \rangle$. Нетерминал $\langle \text{Block} \rangle$ является продуктивным, поскольку повторение $\{ \langle \text{Statement} \rangle \}$ может не выполняться: $\langle \text{Block} \rangle \rightarrow "\{ \}"$. Следовательно, продуктивен и $\langle \text{Body} \rangle \rightarrow \langle \text{Block} \rangle$.

Наконец, для начального символа $\langle \text{While} \rangle$ можно построить, конечный вывод $\langle \text{While} \rangle \rightarrow \text{while} (\langle \text{Condition} \rangle) \langle \text{Body} \rangle$.

Таким образом, для каждого из 16 нетерминальных символов существует хотя бы одна конечная терминальная цепочка. Следовательно, непродуктивные нетерминалы в грамматике отсутствуют.

Совместно с результатами проверки достижимости это позволяет сделать вывод об отсутствии в грамматике бесполезных нетерминальных символов.

2.2.3 Проверка ϵ -продукций

Под ϵ -продукцией понимается продукция вида $A \rightarrow \epsilon$, где ϵ обозначает пустую цепочку.

В исходной грамматике $G[\langle \text{While} \rangle]$ явные ϵ -продукции отсутствуют. Ни один из 16 исходных нетерминалов не способен полностью породить пустую цепочку.

Вместе с тем грамматика записана в EBNF и содержит конструкции повторения $\{ \dots \}$ и необязательные конструкции $[\dots]$. Такие конструкции допускают отсутствие соответствующего фрагмента и при преобразовании грамматики в обычную BNF приводят к появлению вспомогательных ϵ -

продукций. Например, правило $\langle \text{Block} \rangle \rightarrow \{ \langle \text{Statement} \rangle \}$ допускает отсутствие операторов внутри блока. При переходе к BNF оно может быть представлено следующим образом:

$$\langle \text{Block} \rangle \rightarrow \{ \langle \text{StatementList} \rangle \}$$
$$\langle \text{StatementList} \rangle \rightarrow \langle \text{Statement} \rangle \langle \text{StatementList} \rangle \mid \epsilon.$$

Следовательно, вспомогательный нетерминал $\langle \text{StatementList} \rangle$ допускает пустую цепочку, однако сам $\langle \text{Block} \rangle$ пустую цепочку не порождает, поскольку терминальные символы "{" и "}" являются обязательными.

Среди исходных нетерминалов грамматики ϵ -порождающих символов нет. Возникновение ϵ -продукций возможно только для вспомогательных нетерминалов, вводимых при раскрытии конструкций EBNF, и является нормальным следствием используемой формы записи.

2.2.4 Проверка левой рекурсии

Левая рекурсия имеет место в том случае, если некоторый нетерминал способен вывести сентенциальную форму, первым символом которой снова является этот же нетерминал. Различают прямую и косвенную левую рекурсию.

Прямая левая рекурсия имеет вид $A \rightarrow A \alpha$. Например, леворекурсивным являлось бы правило $\langle \text{Arithmetic} \rangle \rightarrow \langle \text{Arithmetic} \rangle "+" \langle \text{Term} \rangle$. Подобные продукции в выбранной грамматике отсутствуют.

При проверке косвенной левой рекурсии необходимо установить, существует ли последовательность переходов вида $A \rightarrow B \dots \rightarrow C \dots \rightarrow A \dots$, при которой возврат к исходному нетерминалу происходит до чтения какого-либо терминального символа.

Рассмотрим основные цепочки левых зависимостей грамматики:

$$\langle \text{Condition} \rangle \rightarrow \langle \text{LogicTerm} \rangle$$
$$\langle \text{LogicTerm} \rangle \rightarrow \langle \text{Relation} \rangle$$
$$\langle \text{Relation} \rangle \rightarrow \langle \text{Arithmetic} \rangle \text{ — при отсутствии необязательного "!"};$$

$\langle \text{Arithmetic} \rangle \rightarrow \langle \text{Term} \rangle$

$\langle \text{Term} \rangle \rightarrow \langle \text{Factor} \rangle$

$\langle \text{Factor} \rangle \rightarrow \langle \text{Identifier} \rangle \mid \langle \text{Number} \rangle \mid \dots$

$\langle \text{Identifier} \rangle \rightarrow \langle \text{Letter} \rangle \dots$

$\langle \text{Number} \rangle \rightarrow \langle \text{Digit} \rangle \dots$

Ни одна из данных цепочек не возвращается к ранее встреченному нетерминалу. В грамматике присутствуют рекурсивные зависимости, однако они не являются леворекурсивными. Например, $\langle \text{Factor} \rangle \rightarrow "(" \langle \text{Condition} \rangle "$ ". Из $\langle \text{Condition} \rangle$ через $\langle \text{LogicTerm} \rangle$, $\langle \text{Relation} \rangle$, $\langle \text{Arithmetic} \rangle$ и $\langle \text{Term} \rangle$ можно вновь перейти к $\langle \text{Factor} \rangle$. Однако перед рекурсивным переходом обязательно считывается терминальный символ "(" . Следовательно, данный цикл не образует левой рекурсии. Аналогичная ситуация наблюдается для вложенных операторов while:

$\langle \text{While} \rangle \rightarrow \text{while} (\langle \text{Condition} \rangle) \langle \text{Body} \rangle$

$\langle \text{Body} \rangle \rightarrow \langle \text{While} \rangle$.

Несмотря на возможность повторного перехода к $\langle \text{While} \rangle$, перед ним уже должна быть распознана терминальная последовательность, начинающаяся с ключевого слова while.

Таким образом, прямая и косвенная левая рекурсия в выбранной грамматике отсутствуют. Это свойство является существенным для реализации синтаксического анализатора методом рекурсивного спуска, поскольку исключает бесконечную последовательность рекурсивных вызовов без продвижения по входной цепочке.

3. Детекция синтаксических ошибок

Синтаксический анализ предназначен для установления соответствия входной последовательности лексем правилам заданной грамматики. Синтаксическая ошибка возникает в том случае, если на некотором этапе разбора очередная входная лексема не может быть использована для продолжения ни одной из допустимых синтаксических конструкций.

Обработка синтаксических ошибок включает два взаимосвязанных процесса: диагностику и нейтрализацию. Под диагностикой понимается определение места возникновения и характера синтаксической ошибки с последующим представлением соответствующего сообщения пользователю. Нейтрализация заключается в восстановлении состояния синтаксического анализатора после обнаруженной ошибки таким образом, чтобы ошибочный фрагмент не препятствовал дальнейшему разбору входной последовательности. В рассматриваемом учебном пособии отмечается отсутствие универсального метода нейтрализации: конкретный способ восстановления определяется структурой грамматики и применяемым алгоритмом синтаксического анализа.

В теории синтаксического анализа применяются несколько основных подходов к восстановлению после ошибок:

1. Аварийное восстановление (режим паники) заключается в пропуске входных символов до достижения заранее выбранного синхронизирующего символа. В качестве таких символов обычно используются терминалы, определяющие границы синтаксических конструкций, например ";;", "}" или ")". Метод отличается простотой и гарантирует продолжение разбора, однако может приводить к пропуску значительного фрагмента входной последовательности.
2. Локальная коррекция предполагает внесение небольшого изменения непосредственно в области возникновения ошибки. Для восстановления разбора может быть виртуально удалён лишний терминал, вставлен

недостающий либо выполнена замена одного терминала другим. Выбранное исправление рассматривается как средство продолжения анализа и не обязательно вносится в исходный текст программы.

3. Использование ошибочных продукций основано на расширении грамматики специальными правилами, описывающими наиболее распространённые ошибочные конструкции. При распознавании такой продукции анализатор может сформировать специализированное диагностическое сообщение. Недостатком подхода является необходимость заранее определить возможные классы синтаксических ошибок.
4. Глобальная коррекция основывается на поиске последовательности преобразований входной цепочки, позволяющей получить синтаксически корректную цепочку с минимальным количеством вставок, удалений или замен терминалов. Такой подход позволяет формализовать процесс исправления, но характеризуется существенно большей вычислительной сложностью.
5. Контекстное восстановление по методу Айронса основывается на состоянии незавершённого синтаксического разбора. После возникновения тупиковой ситуации анализируется структура недостроенных частей синтаксического дерева, удаляются терминалы, препятствующие продолжению разбора, и определяется точка, в которой оставшаяся часть входной последовательности может быть вновь согласована с грамматикой.

Таким образом, задача обработки синтаксической ошибки не ограничивается её обнаружением. Для получения полной диагностической информации необходимо после локализации ошибки восстановить синтаксический разбор и продолжить анализ оставшейся входной последовательности.

3.1 Метод Айронса

Метод Айронса представляет собой способ нейтрализации синтаксических ошибок при нисходящем синтаксическом анализе. Его основная идея заключается в локализации ошибочной части синтаксической структуры и восстановлении разбора с использованием контекста, сформированного к моменту возникновения ошибки. В [1] данный подход описывается как локализация и отсечение «больных» кустов дерева разбора. При возникновении тупиковой ситуации символы, которые не позволяют продолжить анализ по правилам грамматики, последовательно отбрасываются до обнаружения позиции, с которой разбор может быть возобновлён.

Пусть в момент обнаружения ошибки входная цепочка имеет вид

$$xju,$$

где x – уже распознанная часть входной последовательности, ju – ещё не разобранный символ, а $j \in VT$ – текущий терминальный символ.

К моменту возникновения ошибки часть синтаксического дерева уже построена, однако некоторые его ветви остаются незавершёнными. Для этих ветвей определяются символы, которые ещё должны быть распознаны для завершения соответствующих синтаксических конструкций. На основании этой информации формируется множество остаточных символов L .

Алгоритм нейтрализации по методу Айронса включает следующие этапы:

1. Определяются недостроенные части синтаксического дерева и формируется множество L , содержащее остаточные символы этих частей.
2. Проверяется головной терминальный символ j непрочитанной цепочки ju . Если данный символ не может быть выведен ни из одного остаточного символа множества L , он отбрасывается. Операция повторяется для следующих входных терминалов до тех пор, пока не будет найден символ, допустимый для продолжения хотя бы одной из незавершённых синтаксических конструкций.

3. После обнаружения допустимого терминала определяется тот неполный куст синтаксического дерева, с которым может быть согласована оставшаяся входная последовательность. Тем самым устанавливается точка восстановления синтаксического анализа.
4. Определяется терминальная цепочка q , необходимая для завершения промежуточных незаконченных синтаксических конструкций. Для каждого неполного куста формируется минимальная последовательность терминалов, необходимая для его завершения, а их конкатенация образует цепочку q .
5. Сформированная цепочка q логически помещается непосредственно перед найденным терминалом j , после чего синтаксический анализ продолжается с восстановленного состояния.

Таким образом, метод Айронса использует два основных механизма нейтрализации: отбрасывание терминалов ошибочного фрагмента и дополнение незавершённых синтаксических конструкций недостающей терминальной последовательностью. При этом уже успешно разобранный часть входной цепочки повторно не анализируется.

4. Программная реализация

Программная реализация анализа входного текста построена по модульному принципу на языке Java, представленному на рисунке 1. Основные функции распределены между классами лексического анализа, синтаксического анализа и формирования результатов диагностики. Пользовательский интерфейс не участвует непосредственно в разборе программы и в данном разделе не рассматривается.

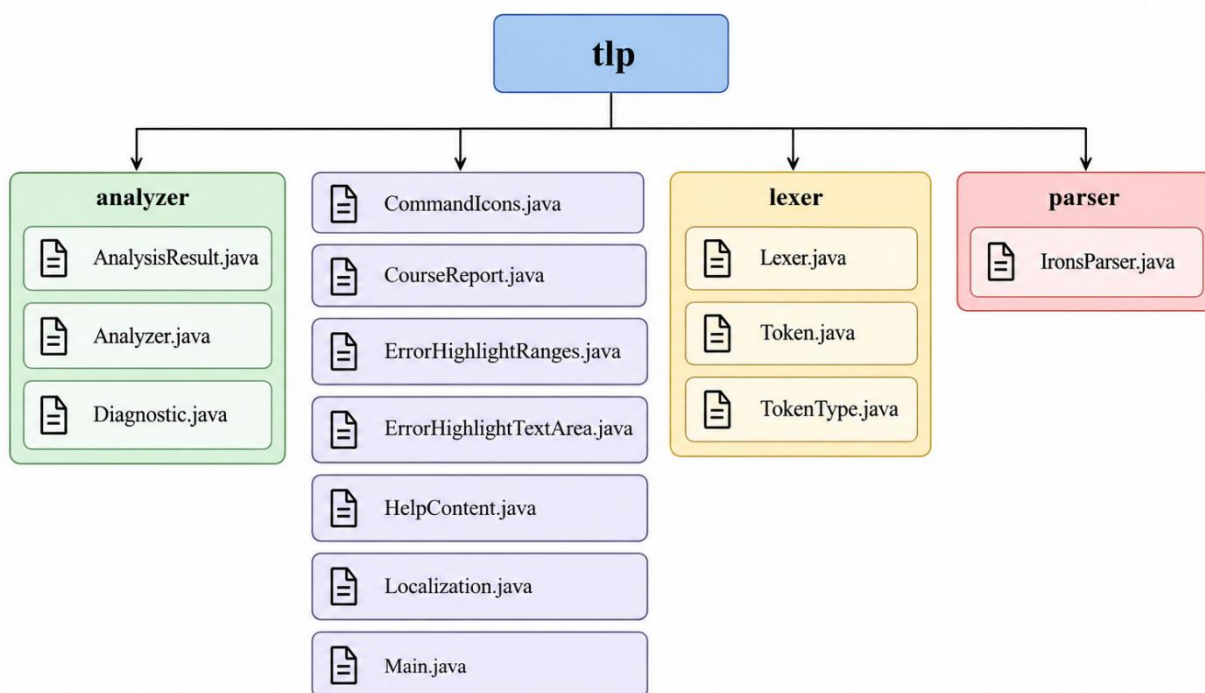


Рисунок 1. Структура проекта while-analyze

Класс `Lexer` выполняет лексический анализ исходного текста. Он последовательно просматривает входную строку и преобразует её в последовательность токенов. В процессе работы распознаются ключевые слова `while`, `break`, `continue`, логические константы `true` и `false`, идентификаторы, целые числа, арифметические и логические операции, операции сравнения и присваивания, скобки и разделители. Составные операции, например `++`, `+=`, `==`, `<=`, `&&`, распознаются как единые лексемы. При обнаружении недопустимого символа формируется диагностическое сообщение, после чего лексический анализ продолжается.

Класс Token представляет отдельную лексему, полученную в результате работы лексического анализатора. Для каждого токена хранятся его тип, исходное текстовое значение и позиция во входной строке. Сохранение позиции необходимо для последующего определения места синтаксической или лексической ошибки.

Перечисление TokenType содержит полный набор типов токенов, используемых анализатором. Использование отдельного типа для каждой лексической конструкции позволяет синтаксическому анализатору работать уже не с отдельными символами, а с формализованной последовательностью лексем.

Класс IronsParser выполняет основной синтаксический анализ последовательности токенов. Внутри класса задаются продукции разработанной грамматики, а конструкции EBNF с повторениями и необязательными элементами представлены с помощью вспомогательных правил. Для выбора необходимой продукции используются множества допустимых начальных токенов, вычисляемые для правил грамматики. При разборе выражений дополнительно анализируется следующий токен, что позволяет различать присваивание, арифметическое выражение, инкремент и декремент, начинающиеся с идентификатора.

В IronsParser также реализована нейтрализация синтаксических ошибок по принципу метода Айронса. При невозможности продолжить разбор анализатор исследует ещё не завершённые элементы синтаксической структуры, определяет допустимые точки восстановления и при необходимости пропускает ошибочную часть входной последовательности. Для незавершённых конструкций вычисляются кратчайшие терминальные последовательности, позволяющие логически завершить их и продолжить разбор. Благодаря этому обнаружение одной ошибки не приводит к немедленному прекращению синтаксического анализа, что позволяет зарегистрировать несколько ошибок за один запуск.

Класс `AnalysisResult` представляет итоговый результат анализа входного текста. Он содержит признак успешного завершения проверки, итоговое сообщение, позицию и длину первой обнаруженной ошибки, а также полный список диагностических сообщений. Таким образом, данный класс служит единым результатом работы всей подсистемы анализа.

Класс `Analyzer` выполняет координацию всех этапов анализа. Сначала исходный текст передаётся лексеру, который формирует последовательность токенов и регистрирует лексические ошибки. Затем полученные токены передаются синтаксическому анализатору `IronsParser`. Диагностические сообщения обоих этапов объединяются и сортируются по позиции во входном тексте. При отсутствии ошибок формируется положительный результат с сообщением о корректности синтаксиса; при наличии ошибок возвращается объект `AnalysisResult`, содержащий сведения обо всех обнаруженных нарушениях.

Таким образом, обработка текста в программе выполняется в последовательности `Lexer` → `Token` → `IronsParser` → `Diagnostic` → `AnalysisResult`, а класс `Analyzer` объединяет перечисленные компоненты в единый процесс анализа. Отдельный модуль семантического анализа в представленной реализации отсутствует: реализованные классы выполняют лексический и синтаксический анализ, а также диагностику и нейтрализацию синтаксических ошибок.

5. Практические примеры

На рисунке 2 представлен пример взаимодействия с редактором While-analyzer

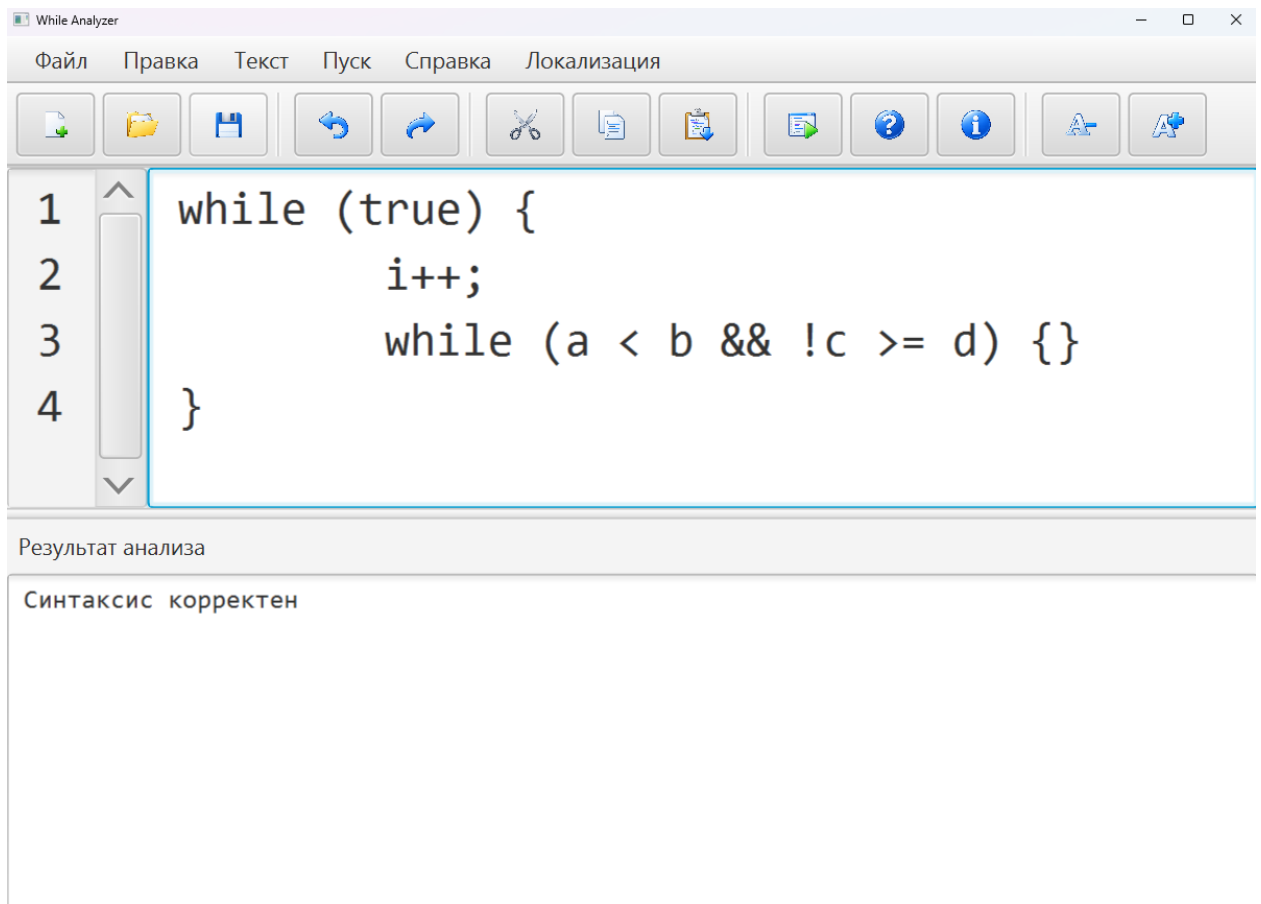


Рисунок 2. Пример выполнения анализа

Заключение

В ходе выполнения работы был изучен процесс построения формальных грамматик и их применения для синтаксического анализа конструкций языков программирования. Для оператора цикла `while` языка C++ была разработана грамматика в расширенной форме Бэкуса–Наура (EBNF), описывающая структуру цикла и все вспомогательные правила.

Разработанная грамматика была классифицирована по Хомскому как контекстно-свободная грамматика типа 2. В рамках её анализа были исследованы достижимость и продуктивность нетерминалов, наличие ϵ -продукций и левой рекурсии. Было установлено, что недостижимые и непродуктивные нетерминалы отсутствуют, а прямая и косвенная левая рекурсия не выявлены.

Также были изучены методы диагностики и нейтрализации синтаксических ошибок. Особое внимание было уделено методу Айронса, основанному на анализе незавершённых частей синтаксической структуры, локализации ошибочного фрагмента и восстановлении разбора с позиции, допускаемой грамматикой.

В практической части был реализован программный анализатор на языке Java, включающий лексический и синтаксический анализ входного текста, диагностику ошибок и механизм их нейтрализации. Были разработаны отдельные программные модули для распознавания токенов, проверки их последовательности по правилам грамматики, формирования диагностических сообщений и объединения результатов анализа.

Кроме того, было разработано кроссплатформенное приложение с графическим интерфейсом на JavaFX, позволяющее вводить исходный текст, запускать его анализ и получать информацию об обнаруженных синтаксических ошибках.

Список использованных источников

1. Шорников Ю.В. Теория и практика языковых процессоров : учеб. пособие / Ю.В. Шорников. – Новосибирск: Изд-во НГТУ, 2004.

Приложение А

Реализация программного комплекса находится в публичном репозитории на сайте github.com: <https://github.com/NeveykoArt/while-analyzer>